

Lab: Kruskal's Algorithm

1. Objective

To implement **Kruskal's Algorithm in C** to find the **Minimum Spanning Tree (MST)** of a weighted undirected graph and determine its minimum total cost.

2. Theory: How It Works

MST: a tree that connects all V vertices using $V-1$ edges with the minimum total weight and no cycle.

- **Kruskal:** sort edges by weight, then pick the smallest edge that does not form a cycle. Stop after $V-1$ edges.

3. Algorithm

1. Enter the number of vertices and edges.
 2. Enter each edge along with its weight.
 3. Sort all edges in increasing order of weight.
 4. Select the smallest edge.
 5. Add it to the MST if it does not form a cycle.
 6. Repeat until $V - 1$ edges have been selected.
 7. Display the selected edges and total MST cost.
-

C Code:

```
#include <stdio.h>

#define INF 9999

int main()
{
    int n;
    int cost[20][20];
    int visited[20] = {0};

    int min, a, b, u, v;
    int ne = 1;
    int minCost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter the cost adjacency matrix:\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);

            if (cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }

    visited[0] = 1;

    printf("\nEdges of Minimum Spanning Tree:\n");

    while (ne < n)
    {
        min = INF;
```

```

    for (int i = 0; i < n; i++)
    {
        if (visited[i])
        {
            for (int j = 0; j < n; j++)
            {
                if (!visited[j] && cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }
    }

    printf("%d - %d : %d\n", a, b, min);

    minCost += min;
    visited[b] = 1;
    ne++;
}

printf("Minimum Cost = %d\n", minCost);

return 0;
}

```

Time Complexity: $O(E \log E)$

The edges are sorted in increasing order of their weights, which takes $O(E \log E)$ time. After sorting, the edges are processed to construct the Minimum Spanning Tree.

Space Complexity: $O(V + E)$

Additional space is required to store the edges and the parent/set information used for cycle detection.

Where:

- V = number of vertices
- E = number of edges

7. Summary

Kruskal's Algorithm is a greedy algorithm used to find the **Minimum Spanning Tree (MST)** of a weighted undirected graph. It sorts all edges in increasing order of weight and repeatedly selects the minimum-weight edge that does not form a cycle. The process continues until **$V - 1$ edges** have been selected.